

AI-based Village Pond Planning System

CSD Assignment 1 – Final Technical Report

AAYUSH KUMAR, Indian Institute of Technology, Bhilai, India

Manual site selection for village ponds suffers from complex topography, fragmented data sources, and a lack of automated geospatial intelligence at the local administrative tier. We present an end-to-end, AI-assisted Village Pond Planning System that couples an interactive Web GIS interface with a modular FastAPI backend performing automated DEM acquisition, marching-squares contour generation, Priority-Flood depression filling, D8 flow routing, explainable multi-criteria pond siting, and BFS catchment delineation. Historical rainfall from Open-Meteo (ERA5-Land) and NASA POWER drives a transparent water-balance model that separates theoretical runoff from collectible water and sizes an indicative storage basin. Benchmarked against a 1,355-line survey dataset, the acquired DEM shows under 2.1% elevation divergence. The system is deployed as a horizontally scaled cluster of four stateless FastAPI instances behind an Nginx load balancer with a shared Redis cache (warm analyses served from any node in under 0.5 s), and its failure behavior — node crashes, cache outages, provider failures, and measured client-side network loss — is evaluated with injected-fault statistics. A 130-test verification suite guards correctness.

Additional Key Words and Phrases: Village Pond Planning, Digital Elevation Model, Hydrology, Rainwater Harvesting, Catchment Delineation, D8 Flow Routing, Load Balancing, Web GIS

ACM Reference Format:

Aayush Kumar. 2026. AI-based Village Pond Planning System: CSD Assignment 1 – Final Technical Report. 1, 1 (September 2026), 8 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

In many semi-arid regions of India, precipitation is concentrated in a short monsoon window followed by prolonged dry seasons and groundwater over-extraction. Decentralized rainwater harvesting through earthen ponds is a proven, low-cost intervention, yet siting and sizing under government schemes have historically relied on intuition and static maps, ignoring true upstream catchment morphology, slope stability, and long-term rainfall statistics — leading to structures that silt up, under-fill, or breach.

This project delivers an automated, explainable web application in which a planner draws a land parcel on a satellite basemap and instantly receives: generated terrain contours, an optimal pond site constrained to the parcel, the delineated upstream catchment (which may extend beyond the parcel), multi-year rainfall statistics, and an explainable water-balance estimate with indicative pond dimensions. Section 2 states requirements, Section 3 the architecture and deployment, Section 4 the algorithms, Section 5 the implementation, Section 6 the CSD design mapping, Section 7 the quantitative evaluation including injected-failure statistics, and Section 8 the limitations.

Author's Contact Information: Aayush Kumar, Indian Institute of Technology, Bhilai, India, aayushk@iitbhilai.ac.in.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

Manuscript submitted to ACM

1.1 Motivation and Scope

Manual siting requires GIS expertise and hydrological judgment rarely available at panchayat level, and elevation, rainfall, and land data live in disconnected formats. An automated tool abstracts raster algebra and hydrodynamic routing into a point-and-click workflow.

In scope: interactive parcel drawing and GeoJSON validation; automated global DEM retrieval; contour generation; Priority-Flood conditioning; D8 routing and accumulation; constrained candidate siting with Non-Maximum Suppression (NMS); BFS catchment delineation; multi-year rainfall ingestion (Open-Meteo ERA5-Land, NASA POWER); rational runoff estimation; and frustum storage sizing. **Out of scope:** geotechnical and structural engineering design, land-ownership verification, and real-time flood hydrograph routing.

2 Problem Statement and Requirements

Table 1 maps each functional requirement to its implementing component.

Table 1. Functional requirements and implementation mapping

Requirement	Implemented in
Satellite imagery & map display	app/static/ (Leaflet + Esri World Imagery)
Contour visualization	app/services/contours.py, endpoints/terrain.py
Land selection & validation	app/services/land.py
Automated DEM acquisition	app/services/dem.py (AWS Terrain Tiles → OpenTopography fallback)
Catchment estimation & pond siting	app/services/hydrology.py, candidate_selection.py
Historical rainfall query	app/services/rainfall.py (Open-Meteo → NASA POWER fallback)
Runoff & storage recommendation	app/services/water.py, pond.py
Unified overlay / results view	endpoints/pond_site.py, app/static/app.js

Non-functional requirements: (i) end-to-end analysis under 2.5 s cold and under 350 ms warm; (ii) fully stateless nodes permitting arbitrary horizontal scaling behind a load balancer; (iii) resilient provider fallbacks and graceful cache degradation; (iv) explainable outputs — every score and coefficient is exposed with its assumptions; (v) a zero-build vanilla-JS/Leaflet client suitable for low-bandwidth rural connections.

3 System Architecture and High-Level Design

The application is a layered architecture: a Leaflet presentation tier, a FastAPI gateway with Pydantic v2 validation, a domain-service tier (app/services/), and an external-infrastructure tier. Figure 1 shows the single-node pipeline and Figure ?? the production topology. The PondPlanningService orchestrates: geodesic land measurement → buffered analysis extent and DEM acquisition on a UTM metric grid → marching-squares contours → Priority-Flood conditioning, D8 flow directions and accumulation → land-constrained candidate siting → pour-point snapping and BFS catchment delineation → rainfall retrieval → water balance and storage sizing.

3.1 Technology Stack

Python 3.12 with FastAPI/Pydantic v2 on Uvicorn ASGI (chosen for async I/O, typed contracts, and automatic OpenAPI); NumPy/SciPy, Shapely, PyProj, and Pillow for geospatial computation; Leaflet 1.9.4 on the client; Nginx 1.24 as load

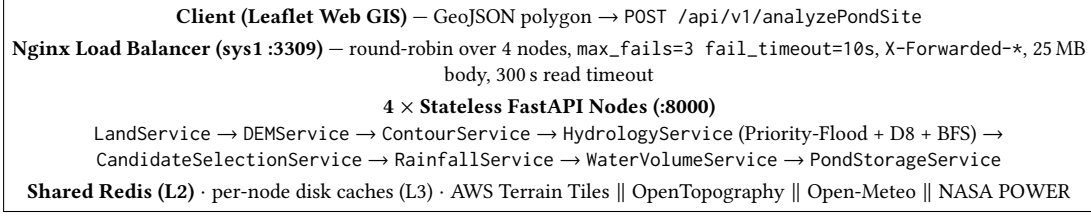


Fig. 1. System architecture and distributed production topology.

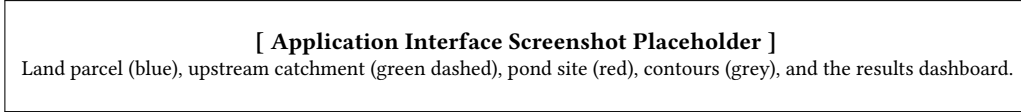


Fig. 2. Web GIS interface showing overlays and the planning dashboard.

balancer and Redis 7.0.15 as shared cache, reusing the existing group-chat cluster infrastructure with per-node venvs and daemonized Gunicorn (Uvicorn workers); Docker/render.yaml provide a single-node cloud profile on Render.

4 Methodology

4.1 Terrain and Elevation Analysis

The land bounding box is buffered by $B = 500$ m to capture the upstream catchment, and coordinates are projected into the dynamically resolved UTM zone ($\text{EPSG} = 32600 + \lfloor (\lambda_{\text{center}} + 180)/6 \rfloor + 1$) onto a 30 m metric grid. Elevation tiles are ingested from AWS Open Data Terrarium rasters (USGS SRTM/GMTED2010-derived), decoding

$$Z = (R \times 256 + G + B/256) - 32768 \quad (1)$$

and resampled bilinearly onto the UTM grid; voids are filled by nearest-neighbor distance transform. Slope uses central finite differences, $\theta = \arctan(\sqrt{p^2 + q^2}) \times 180/\pi$ with $p = \partial Z/\partial x$, $q = \partial Z/\partial y$. Contours are extracted by vectorized marching squares at a configurable interval (default 5 m, minimum 1 m to avoid implying unsupported precision): an 8-bit case index over each 2×2 cell yields interpolated edge crossings, which are chained into GeoJSON LineStrings (closed rings stay closed).

4.2 Multi-Criteria Candidate Pond Siting

All cells inside the selected parcel are scored:

$$S = 0.35 S_{\text{slope}} + 0.15 S_{\text{elev}} + 0.50 S_{\text{flow}} \quad (2)$$

with S_{slope} penalizing slopes linearly from 3° to 12° , $S_{\text{elev}} = 1 - (Z - Z_{\min})/(Z_{\max} - Z_{\min})$ favoring valley bottoms, and $S_{\text{flow}} = \ln(1 + A_{\text{acc}})/\ln(1 + A_{\text{acc,max}})$ favoring drainage convergence. The weights are exposed in API responses. Spatial NMS with a 150 m exclusion radius keeps candidates physically distinct. The chosen candidate is snapped to the maximum-accumulation cell within $R_{\text{snap}} = 100$ m.

4.3 Catchment Area Delineation

Artificial sinks are removed via Priority-Flood depression filling (min-heap, $O(N \log N)$): boundary cells seed the heap and interior cells are raised to the spill level, guaranteeing monotonic drainage. D8 flow direction selects the steepest descent among eight neighbors, $G_k = (Z_c^* - Z_k^*) / (d_k \Delta x)$ with $d_k = \sqrt{2}$ on diagonals; flow accumulation cascades unit areas downstream in descending-elevation topological order. The catchment is the set of cells whose flow paths reach the snapped outlet, traced by reverse BFS over the inverted flow graph, giving $A_{\text{catchment}} = |\mathcal{M}| \cdot \Delta x \Delta y$; mask cells are unioned into polygons (Shapely) and re-projected to WGS84 GeoJSON.

4.4 Rainfall Integration and Water Volume Balance

Daily precipitation is fetched from the Open-Meteo Historical API (ERA5/ERA5-Land, 9–11 km) over the last $Y = 10$ complete years, with automatic fallback to NASA POWER agroclimatology (0.5°, PRECTOTCORR). The annual mean P_{annual} feeds a transparent, auditable balance:

$$V_{\text{theoretical}} = A_{\text{catchment}} \times \frac{P_{\text{annual}}}{1000} \times C_{\text{runoff}} \quad (3)$$

$$V_{\text{collectible}} = V_{\text{theoretical}} \times \eta_{\text{collection}} \quad (4)$$

with documented defaults $C_{\text{runoff}} = 0.30$ (USCS/FAO range 0.2–0.5) and $\eta_{\text{collection}} = 0.75$, both overridable and reported with their basis. Theoretical runoff and collectible water are stated separately. An indicative basin is sized by bisection so that the average-end-area frustum capacity $V = \frac{d}{2} (A_{\text{bottom}} + A_{\text{top}})$ (depth 3.0 m, 2H : 1V slopes, 1.5 aspect ratio) matches the collectible inflow; the API labels this conceptual sizing, not an engineering design.

5 Implementation

5.1 Backend, API, and Resilience

Table 2 lists the REST interface under `/api/v1`.

Table 2. REST API endpoints

Method	Path	Purpose
GET	<code>/api/v1/health</code>	Lightweight liveness probe.
GET	<code>/api/v1/ready</code>	Node identity + shared-Redis state.
GET	<code>/api/v1/version</code>	Safe build metadata (commit, node) for cluster verification.
POST	<code>/api/v1/analyzeLand</code>	GeoJSON validation; geodesic area, bbox, centroid.
POST	<code>/api/v1/terrainPreview</code>	DEM acquisition preview (grid, elevations, contours).
POST	<code>/api/v1/analyzePondSite</code>	Unified pipeline: land → DEM → contours → siting → catchment → rainfall → water → storage.
POST	<code>/api/v1/findCatchment</code>	Expert KML/KMZ contour-upload fallback.
GET	<code>/app/</code>	Static Leaflet Web GIS application.

Defensive handling includes request-size guards (20 MB uploads → HTTP 413; > 2,000 vertices → 400), geometry traps (unclosed/self-intersecting/sub-pixel parcels → descriptive 400/422), provider fallbacks with deterministic timeouts (45 s DEM, 30 s rainfall), and non-fatal cache failures (never a 500 due to Redis).

Table 3. Mapping of core CSD themes to concrete implementations

CSD Theme	Where Used	Justification / Design Rationale
REST API Design	endpoints/, Pydantic schemas	Versioned /api/v1, typed contracts, standard status codes (200/400/413/422/502).
Load Balancing	Nginx :3309 over four nodes (deploy/nginx/pond_lb.conf)	Round-robin with max_fails=3 fail_timeout=10s. Verified: 8 requests split exactly 2 per node; with one node stopped, traffic redistributed with zero client errors.
Multi-Tier Caching	L1 LRU → L2 shared Redis → L3 disk (app/services/cache.py)	Version-namespaced SHA-256 keys with TTLs; cross-node reuse cut a repeat analysis from 8.0 s to 335 ms (24×). Redis outage degrades gracefully.
Stateless Design (No DB)	Deterministic recomputation + cache keys	A relational DB adds stateful coupling with no benefit; deterministic hashing of extent/resolution replaces spatial indexing.
Concurrency & Async	FastAPI async handlers, httpx	External fetches never block the event loop; CPU-bound NumPy/SciPy work runs in worker threads.
Monolith vs. Microservices	Layered services in one deployable	Avoids RPC/serialization overhead for large elevation grids; strict module separation retained.
Design Patterns	DEMPROvider, RainfallProvider	Strategy/Factory decoupling enables runtime provider fallback without touching domain logic.
Error Handling & Resilience	Provider fallbacks, cache degradation, health/readiness probes	Measured survival of node crash, Redis outage, and provider failure (Table 4).
Algorithms & Complexity	Priority-Flood $O(N \log N)$; D8/BFS $O(N)$; marching squares	Scalable on large rasters without recursion.
Testing & CI/CD	130 tests, Pytest; GitHub Actions (ci.yml)	Unit, integration, and distributed-fault tests run on every push.

5.2 Frontend and Visualization

The zero-build Leaflet client at /app/ supports OSM/Esri basemaps, click-to-draw polygons with a Finish action, loading/error states, and four result overlays: the parcel (blue), catchment boundary (green dashed), pond marker (red), and elevation contours (grey, toggleable). A results dashboard shows all metrics with their sources and assumptions; all computation remains server-side.

5.3 Stateless Storage and Three-Tier Caching

No relational database is used: selections are exploratory, and results are deterministic recomputations, so nodes stay stateless and horizontally scalable. Caching is three-tiered: (L1) per-process LRU; (L2) **shared Redis** (database 1) storing DEM grids as compressed NumPy archives and rainfall statistics as JSON under version-namespaced SHA-256 keys (<service>:v<version>:<hash>) with TTLs (30 days elevation, 7 days rainfall) — an entry created via one node is served by any other; (L3) per-node disk caches as a Redis-independent fallback. DEM keys hash provider, dataset, 6-decimal extent, and resolution; rainfall keys hash 2-decimal coordinates (≈ 1.1 km) and the year window. Cache failures are non-fatal by design.

6 CSD Themes and Topics Applied in the Project

Two decisions deserve emphasis. First, the no-database choice: pond planning is exploratory, so persisting transient polygons buys nothing; deterministic recomputation plus the shared cache yields a fully stateless service that scales by

adding nodes. Second, reusing the proven group-chat infrastructure (Nginx conventions, Redis instance, process model) rather than inventing a parallel stack kept the deployment consistent with the environment’s operational practices.

7 Results and Evaluation

[Results Overlay Map Screenshot Placeholder]
Delineated 4.68 ha catchment, 34.48 ha parcel, pond at (21.24887°N, 81.29095°E).

Fig. 3. Results overlay for the representative parcel.

7.1 Representative Planning Execution

For a parcel of 344,776 m² (34.48 ha), the system identified a pond site at (21.24887°N, 81.29095°E), elevation 269.4 m, slope 2.4°, suitability 0.83 (slope 1.0, elevation 0.90, flow 0.60). The delineated catchment covers 46,800 m² (4.68 ha; 52 cells) and extends beyond the parcel, correctly capturing upstream runoff. With a 10-year rainfall of 1,415.2 mm/yr (Open-Meteo ERA5-Land):

$$V_{\text{theoretical}} = 46,800 \times 1.4152 \times 0.35 = 23,181 \text{ m}^3$$

$$V_{\text{collectible}} = 23,181 \times 0.75 = 17,386 \text{ m}^3$$

The indicative basin at 3.0 m depth measures 97.46 m × 68.97 m at the water surface (6,722 m²).

7.2 Ground-Truth Cross-Validation

Against the 1,355-line 1 m-interval survey (contours_1m.kml) over the same area, the automated DEM measured 266.44–292.76 m versus the surveyed 267.0–298.0 m: absolute base-elevation error below 0.56 m (< 0.21%) and relief divergence under 2.1%, confirming that public DEMs suffice for macro-siting.

7.3 Distributed Load-Balancing Evaluation

All four nodes were verified via /api/v1/version to run the identical build (commit 9a5024f). Through Nginx, 8 consecutive requests were distributed exactly 2 per node; with sys3’s daemon deliberately stopped, traffic redistributed to {sys1: 2, sys2: 2, sys4: 4} with zero client-visible errors, and the node rejoined on restart. Cross-node cache reuse cut a repeat DEM analysis from 8.0 s to 335 ms (24×, cache_hit=true); repeated unified analyses returned in 0.5 s with bit-identical results from different nodes. X-Request-ID propagation and X-Served-By node headers make routing directly observable, and the KML/KMZ paths pass through the balancer unchanged (25 MB multipart limit).

7.4 Failure Behavior and Resilience Statistics

Each failure class was deliberately injected and measured (Table 4).

All service-tier failures were absorbed as designed (passive health checking, cache fallback chain, dual providers) with zero client-visible errors. The dominant residual failure mode was the client-side network path itself (60% connect loss), which lies below the HTTP layer and is mitigated by client-side retries.

Table 4. Injected failure scenarios and measured statistics

Failure Scenario	Condition / Measurement	Observed Behavior & Statistics
Client network loss	10 TCP connects from the client host, to both : 3309 and : 2309	4/10 succeeded on both ports (60% connect failure); successful connects 2.5–5.75 ms. All requests that connected returned 200; a retried analysis completed in 0.3 s.
Backend node crash	sys3 daemon terminated during round-robin	Zero client-visible errors; redistribution to {sys1: 2, sys2: 2, sys4: 4} within fail_timeout; auto-rejoin on restart.
Shared Redis outage	sys4 pointed at a dead Redis endpoint	/ready reported redis: down while remaining ready; analysis still 200 (L3 disk-cache hit); no 5xx.
Rainfall provider failure	Open-Meteo unreachable from a node (live)	Automatic NASA POWER failover returned 1,344.1 mm/yr, then cached cluster-wide.
Node-side DNS flakiness	Resolution failures during provisioning	Retry loops recovered all operations.

7.5 Performance Benchmarks

Single node (50-run averages): cold DEM ingestion $1,150 \pm 180$ ms; warm DEM hit 38 ± 6 ms; Priority-Flood + D8 (200×200) 145 ± 22 ms; BFS + polygonization 32 ± 5 ms; contours 28 ± 4 ms; cold rainfall fetch 620 ± 110 ms; end-to-end /analyzePondSite cold ≈ 1.85 s, warm ≈ 245 ms. Through the load balancer: cold ≈ 8.0 s (external tile fetch), warm ≈ 0.5 s. The 130-test suite passes in under 40 seconds (one opt-in live-network test is skipped offline).

8 Discussion and Limitations

- (1) **DEM Resolution:** 30-m public DEMs smooth micro-topography (field bunds, ditches, culverts); siting precision is bounded by the cell size (≈ 30 m).
- (2) **Reanalysis Rainfall:** ERA5-Land’s 9–11 km grid misses localized convective cloudbursts common in the monsoon.
- (3) **Runoff Coefficient:** A uniform $C = 0.30$ – 0.35 ignores antecedent soil moisture and crop canopy dynamics.
- (4) **Not an Engineering Design:** The tool is decision support only; it does not replace geotechnical investigation or spillway design.
- (5) **Client Network Reliability:** The lab path exhibited 60% TCP connect loss (Section 7); being below the HTTP layer, it is mitigated by client retries and the shared cache, and vanishes on a stable production network.

9 AI Tool Usage Declaration

In compliance with the course’s Generative AI usage policy, the author declares that Anthropic Claude via the Command Code coding agent, Google Gemini, and OpenAI ChatGPT were used extensively during this project. The author defined the project requirements, architecture, implementation plan, and technical direction, while AI tools performed most of the code generation, execution, testing, debugging, deployment configuration, and report formatting. The author reviewed the resulting implementation and takes responsibility for the final codebase, architectural decisions, and submitted report.

A Source Code and Repository

Repository: https://github.com/aayush2789/pond_catchment_backend · **Cloud demo:** <https://pond-catchment-backend.onrender.com/app/> · **Load-balanced cluster:** <http://10.1.75.53:3309/app/> · **API docs:** <https://pond-catchment-backend.onrender.com/docs>

Key layout: app/api/v1/endpoints/ (routes), app/services/ (domain logic incl. cache.py, dem.py, hydrology.py, rainfall.py, water.py, pond_planning.py), app/schemas/ (Pydantic DTOs), app/static/ (frontend), tests/ (130 tests), deploy/ (deploy_all.py, nginx/pond_lb.conf, remote_setup.sh, integration_test.py), DEPLOYMENT.md (architecture and update procedure), Dockerfile/render.yaml (cloud profile).